

Sri Lanka Institute of Information Technology



Control Theory (IE2074) **Final Assignment**

Design and Implementation of PID-Based DC Motor Speed Control using ESP32 and L298N Motor Driver

Group Number: 09

Members: IT24610806 - R.A.T.P.Obeysekara

IT24101346 - K.T.W.Gunasekara

Submission Date: 1st May 2026

Table of Contents

1. Introduction.....	5
2. Design Methodology.....	6
2.1 Closed-Loop System Architecture.....	6
2.2 Working Principle.....	7
2.3 Mathematical Representation of PID.....	8
2.3.1 Continuous-Time PID Law	8
2.3.2 Discrete-Time Implementation.....	9
3. Component Selection.....	10
3.1 Microcontroller: ESP32 vs. Arduino Uno	11
3.2 Motor Driver: L298N	11
3.3 Power Supply: Rechargeable 12 V Battery and Boost Converter	12
3.4 DC Motor with Encoder	12
4. Results & Testing.....	13
4.1 Circuit Diagram	13
4.2 Source Code (Arduino IDE)	15
4.2.1 Pin and Parameter Configuration	15
4.2.2 Encoder Interrupt Service Routine	16
4.2.3 Setup Function.....	17
4.2.4 Main PID Control Loop.....	18
4.2.5 GUI Part.....	20
4.3 PID Tuning Values	22
4.4 System Response Graphs.....	23
4.5 Graphical User Interface (GUI)	24
4.6 Performance Analysis	25

The performance of the PID-controlled system was evaluated based on standard control parameters:	25
• Rise Time: ~2 seconds.....	25
• Settling Time: ~2–2.5 seconds	25
• Overshoot: <5%.....	25
• Steady-State Error: ≈ 0 RPM.....	25
The system demonstrates stable behavior with minimal oscillations and fast convergence to the desired speed. The integral component ensures zero steady-state error, while the derivative component helps reduce overshoot.	25
5. Discussion	26
5.1 Challenges Faced	26
5.2 Improvements	27
6. Conclusion	29
7. References.....	30

Table of Figures

Figure 1: Block Diagram- Closed Loop System Architecture.....	6
Figure 2: Wiring diagram illustrating ESP32 to L298N and Encoder connections.....	13
Figure 3: System step response showing rise time and steady-state error.....	23
Figure 4: GUI Screen	24

Table of Tables

Table 1: Defining Terms of Continuous-Time PID Law	8
Table 2: Component Selection.....	10
Table 3: ESP32 Pin Configuration.....	14
Table 4:PID Tuning Values	22

1. Introduction

The PID controller is the most used feedback control method in industrial automation and embedded systems, which is utilized in more than 95% of feedback control loops (Åström & Hägglund, 1995). Its continued success is due to its simplicity, reliability, and capability to provide a satisfactory response over a broad range of plant dynamics without the requirement of a mathematical model of the plant.

This report explains how to design, implement, and test a PID controller that manages the speed of a DC motor using an ESP32 Dev Kit V1 microcontroller. To control the motor, the controller utilizes the L298N H-Bridge Motor Driver IC to connect the motor with the incremental encoder, enabling real-time tracking of its speed. This is implemented through a control process whose sampling frequency is 10 Hz (100 ms). It operates based on a set of 150 RPM.

Objective:

The objective of the project is to design and build a discrete-time proportional-integral-derivative (PID) controller for an ESP32 microcontroller to control the speed of the DC motor at 150 RPM and analyze its performance parameters like rise time, settling time, percentage overshoot, steady-state error, and stability.

The rest of the report is organized as follows: In Section 2, the design approach, which includes the closed-loop diagram of the system along with the derivation of the discrete PID equation, is explained. The justification for the choice of components is included in Section 3. Section 4 consists of the circuit design, complete source code, PID tuning experiment, and response of the system.

2. Design Methodology

2.1 Closed-Loop System Architecture

The system is designed as a closed-loop feedback control system. This is shown through the block diagram of the system in Figure 1. The reference input signal (set point, r) is compared against the actual output signal of the plant (motor speed, y) to get the error signal (e). The error signal is fed into the PID controller, which then gives out the control signal (u), PWM duty cycle value. This control signal is passed on to the L298N Motor Driver, which in turn passes a proportional voltage to the DC motor.

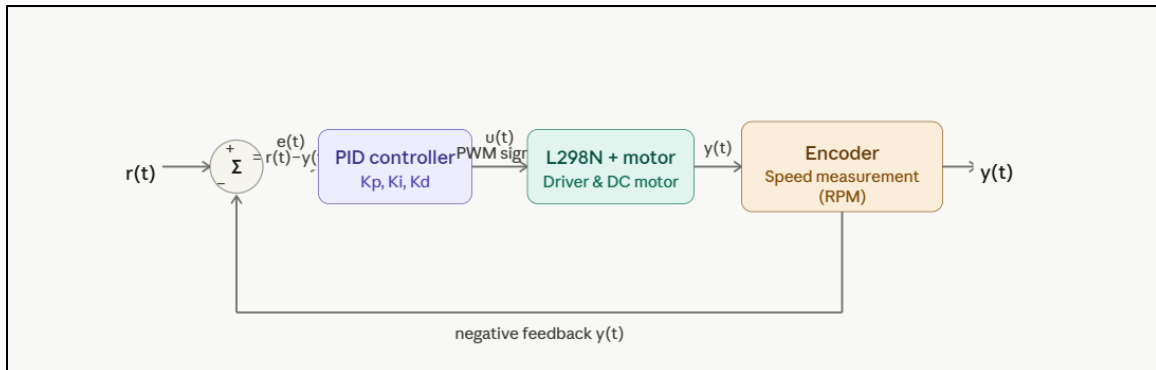


Figure 1: Block Diagram- Closed Loop System Architecture

Signal	Symbol	Description
Reference / Setpoint	$r(t)$	Desired motor speed
Error Signal	$e(t) = r(t) - y(t)$	Difference between desired and actual speed
Control Signal	$u(t)$	PID output, sent as PWM to motor driver
Actual Speed	$y(t)$	Measured motor speed in RPM (feedback)

2.2 Working Principle

The system operates on a closed-loop feedback mechanism. The encoder provides pulses that indicate rotation speed. The ESP32 processes these pulses to compute the current RPM. The error ($e(t)$) is defined as the difference between the Target RPM and the Measured RPM.

The control loop performs the following steps every 100 ms:

1. **Pulse counting from encoder:** The ISR for GPIO 34 updates a volatile variable on each rising edge of Encoder Channel A, increasing or decreasing according to the position of Channel B. Atomic interrupts are disabled while the counter is read and reset.
2. **Speed calculation:** The RPM is calculated from the number of pulses captured during the 100 ms interval using the formula $RPM = (\text{count}/PPR) \times (60/dt)$, where $PPR = 200$ pulses per revolution and $dt = 0.1$ s.
3. **Calculate error:** $e[k] = \text{Setpoint} - \text{Speed}$. Given that the setpoint is 150 RPM and the speed is 130 RPM, the error is +20 RPM, meaning the control system needs to boost the output signal.
4. **PID calculation:** Three components are determined and added: The P component acts on present error, the I component sums up past errors to overcome steady-state error, and the D component predicts future errors based on the change in error.
5. **Anti-windup implementation:** The integrator value is capped between -200 and 200 to avoid integrator windup during output saturation, such as during motor initialization.

6. **PWM generation:** The generated PID output is limited to $[0, 255]$, and then it is set on GPIO 25 using `ledcWrite()` to control the ENA input of the L298N, which regulates the average motor voltage.

2.3 Mathematical Representation of PID

2.3.1 Continuous-Time PID Law

The ideal continuous-time PID control law relates the control output $u(t)$ to the error signal $e(t)$ as:

$$\text{Eq. 1: } u(t) = K_p \cdot e(t) + K_i \cdot \int_0^t e(\tau) d\tau + K_d \cdot [de(t) / dt]$$

Where K_p , K_i , and K_d are proportional, integral, and derivative gains, respectively. Each term contributes a distinct aspect of the control action:

Table 1: Defining Terms of Continuous-Time PID Law

Term	Gain	Effect on Control Response
Proportional (P)	K_p	Drives output proportionally to current error. Increases response speed but alone cannot eliminate steady-state error (offset).
Integral (I)	K_i	Accumulates past errors. Eliminates steady state offset but can cause overshoot and integral windup if unconstrained.
Derivative (D)	K_d	Reacts to rate of change of error. Provides damping, reduces overshoot, but amplifies high-frequency measurement noise.

2.3.2 Discrete-Time Implementation

Because the PID control law is computed at discrete time intervals, the continuous control law is converted to a discrete control law using the backward Euler method (rectangular approximation) with sampling period $T=0.1$ seconds:

$$\text{Eq. 2: } u[k] = K_p \cdot e[k] + K_i \cdot T \cdot \sum e[i] + K_d \cdot (e[k] - e[k-1]) / T$$

Where the integral is computed using an accumulated sum, while the derivative is computed using the first backward difference approximation. In the code snippet, T (dt) is based on the elapsed time between consecutive calls, calculated using `millis()`, rather than a constant value. This approach increases the robustness of the implementation against loop delay jitter.

$$\text{Eq. 3 (anti-windup): } I[k] = \text{clamp}(I[k-1] + e[k] \cdot dt, -200, +200)$$

Anti-windup clipping (Eq. 3) is used to bound the integral term to a certain range (-200, 200). Otherwise, without the anti-windup technique, the integral term may grow unbounded, especially when the actuator is saturated at its upper and lower bound limits. The lack of anti-windup would result in severe overshooting and delayed settling behavior.

3. Component Selection

All hardware elements were chosen based on the needs of the closed loop system in terms of electronics and computations. All hardware elements are listed below along with their specifications and reasons for their choice.

Table 2: Component Selection

Component	Specification	Role in System	Justification
ESP32 Dev	Xtensa LX6, 240 MHz, 3.3 V I/O, LEDC PWM, 34 interrupt-capable GPIO	Microcontroller — runs PID algorithm, generates PWM, reads encoder	LEDC hardware PWM avoids CPU-blocking software PWM; GPIO 34/35 are input-only (safe for encoder); FreeRTOS-capable for future enhancement
L298N H-Bridge Module	Dual channel, 2 A/ch, 5–35 V motor supply, onboard 5 V reg, integrated flyback diodes	Motor driver — amplifies logic-level PWM to motor-level current	Direct ENA PWM input; built-in protection diodes prevent back-EMF damage to ESP32; onboard regulator can power logic (optional)
DC Motor + Encoder	6–12 V DC brushed, 20 PPR incremental quadrature encoder (Hall-effect or optical)	Actuator + speed sensor — converts electrical energy to rotation and provides shaft speed feedback	Quadrature encoding allows direction detection and relative speed measurement; directly compatible with ESP32 interrupt inputs

XL6009 Step-Up Module + 9 V Battery	Input: 9 V battery; Output: 12 V DC (XL6009 adjusted), ≥ 1 A rated	Motor power supply — XL6009 boosts 9 V battery to regulated 12 V for the L298N VCC	Within the L298N operating voltage range (5–35 V); sufficient to drive small DC motors at the design speed
Decoupling Capacitors (recommended)	100 nF ceramic across motor terminals	EMI suppression	Motor brush noise generates RF interference that can corrupt encoder signals; capacitors reduce this coupling

3.1 Microcontroller: ESP32 vs. Arduino Uno

Although the Arduino Uno with the chip ATmega328P operating at 16 MHz was taken into consideration, it was not chosen for this assignment. Comparing the microcontrollers, this decision might have been caused by the following. First, the ATmega328P has two external interrupt pins (INT0, INT1), whereas the ESP32 allows an interruption from any GPIO pin. Secondly, hardware PWM is restricted to the use of specific pins and predefined frequencies for the Uno, whereas the ESP32 can offer up to 16 channels using LEDC hardware. Thirdly, Arduino Uno is a device that works at 5V logic; thus, an appropriate level shifter is required when interacting with 3.3V components, including the encoders.

3.2 Motor Driver: L298N

Of all the available motor drivers, the L298N was chosen over other options like the L293D driver, which had lower current capacity than the former and did not have internal diodes, and the DRV8833 driver, which required the I2C/SPI protocol for communication and was harder to control using PWM signals. In addition, the L298N dual full-bridge configuration made it easier to control the direction and speed of the motor independently.

3.3 Power Supply: Rechargeable 12 V Battery and Boost Converter

A rechargeable 12 V battery was selected as the main motor power source. However, during testing, the battery did not maintain a full 12 V output due to aging and charge degradation. Its measured voltage was around 10.5 V, and it discharged quickly when connected to the motor load. This caused the motor speed to drop and made the PID response less stable, because the available motor voltage changed during operation.

To overcome this issue, a DC-DC boost converter was used to step the battery voltage back up to a more stable 12 V supply for the L298N motor driver. This helped the motor receive a more consistent driving voltage and improved the reliability of the speed control system.

3.4 DC Motor with Encoder

The motor used in this project was a 12 V DC geared motor with a rated speed of 300 RPM. This motor was suitable for the project because the target speed was 150 RPM, which is within the controllable operating range of the motor. Since the motor speed was higher than the required setpoint, the PID controller could reduce the PWM duty cycle and regulate the motor speed accurately.

4. Results & Testing

4.1 Circuit Diagram

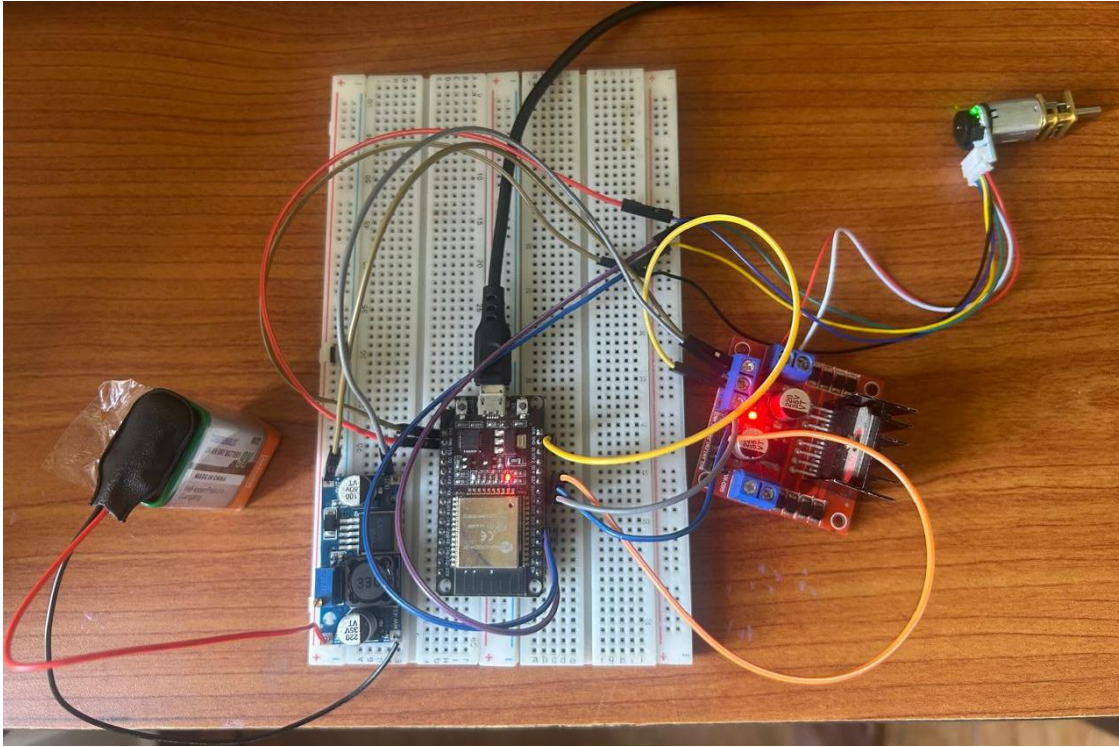


Figure 2: Wiring diagram illustrating ESP32 to L298N and Encoder connections.

The hardware section employs the ESP32 microcontroller for executing the PID control algorithm and for sending data from the PID to the dashboard using the serial port. The ESP32 will send the PWM control signal and the direction control signal to the L298N motor controller. In addition, it is responsible for obtaining speed feedback from the motor encoder and comparing it with the desired speed. Finally, the jumper wires provide power, ground, control, and feedback connections, whereas the battery powers the entire circuit.

Table 3: ESP32 Pin Configuration

Signal	ESP32 Pin	Wire Colour	L298N / Peripheral Pin	Notes
PWM Speed	GPIO 25	Yellow	ENA	1 kHz, 8-bit (0–255 duty cycle)
Direction A	GPIO 26	Blue	IN1	Set HIGH for forward rotation
Direction B	GPIO 27	Green	IN2	Set LOW for forward rotation
Common GND	GND	Black	GND (L298N)	Also connect XL6009 GND and battery – here
Encoder VCC	3.3 V	Red	Encoder VCC	Max 3.3 V (ESP32 I/O level)
Encoder GND	GND	Black	Encoder GND	Shared ground rail
Encoder A (IRQ)	GPIO 34	White	Signal A	Interrupt on RISING edge
Encoder B (DIR)	GPIO 35	Grey	Signal B	Polled inside ISR
Motor +	—	Red	OUT1 → Motor M+	From L298N output
Motor –	—	Black	OUT2 → Motor M–	From L298N output
XL6009 Output +	—	Red	VCC (12 V pin)	12 V regulated (XL6009 output from 9 V battery)
XL6009 / Battery –	—	Black	GND	Common ground

4.2 Source Code (Arduino IDE)

The entire firmware is written in C++ programming language using the ESP32 Arduino Core v3.x in Arduino IDE. The coding process is done in five main modules, namely, configuration, constant values, global variables, encoder ISRs, setup () and main loop.

4.2.1 Pin and Parameter Configuration



```
it Sketch Tools Help
ESP32 Dev Module
esp_dash_n.ino  html_page.h
1  // =====
2  // PID DC Motor Speed Control - with WiFi Dashboard
3  // ESP32 DevKit + L298N + DC Motor with Encoder
4  // Arduino ESP32 Core v3.x Compatible
5  // -----
6  // STA MODE - connects to an existing WiFi network (e.g. school)
7  // =====
8
9  #include <WiFi.h>
10 #include <WebServer.h>
11 #include <WebSocketsServer.h> // arduinoWebSockets by Markus Sattler
12 #include "html_page.h"
13
14 // ----- WiFi STA credentials -----
15 // *** CHANGE THESE to match your school WiFi ***
16 const char* wifi_ssid = "Oppo A12";
17 const char* wifi_password = "them12003";
18
19 // ----- Server objects -----
20 WebServer httpServer(80);
21 WebSocketsServer wsServer(81);
22
23 // --- Pin Definitions ---
24 #define ENA_PIN 25
25 #define IN1_PIN 26
26 #define IN2_PIN 27
27 #define ENCODER_A 34
28 #define ENCODER_B 35
29
30 // --- Encoder Settings ---
31 #define PPR 200 // Change to match your encoder
32
```

```
Edit Sketch Tools Help
ESP32 Dev Module

esp_dash_n.ino  html_page.h
33 // --- PID Parameters ---
34 float Kp = 1.2;
35 float Ki = 0.10;
36 float Kd = 0.05;
37
38 // --- Setpoint ---
39 float setpoint = 150.0; // Target RPM
40
41 // --- PID Variables ---
42 float actualSpeed = 0.0;
43 float error = 0.0;
44 float prevError = 0.0;
45 float integral = 0.0;
46 float derivative = 0.0;
47 float pidOutput = 0.0;
48 float pidOutput_prev = 0.0;
49
50 // --- Anti-windup limits ---
51 float integralMax = 200.0;
52 float integralMin = -200.0;
53
54 // --- Encoder ---
55 volatile long encoderCount = 0;
56
57 // --- Timing ---
58 unsigned long lastTime = 0;
59 const unsigned long sampleTime = 100; // ms
60 unsigned long lastIpPrint = 0;
61 unsigned long lastPidPrint = 0;
62
```

4.2.2 Encoder Interrupt Service Routine

```
63 // =====
64 // INTERRUPT
65 // =====
66 void IRAM_ATTR encoderISR() {
67     if (digitalRead(ENCODER_B) == HIGH) {
68         encoderCount++;
69     } else {
70         encoderCount--;
71     }
72 }
73
```


4.2.3 Setup Function

```
108 // =====
109 // SETUP
110 // =====
111 void setup() {
112     Serial.begin(115200);
113     delay(500);
114
115     // --- Motor & Encoder ---
116     pinMode(IN1_PIN, OUTPUT);
117     pinMode(IN2_PIN, OUTPUT);
118     pinMode(ENCODER_A, INPUT_PULLUP);
119     pinMode(ENCODER_B, INPUT_PULLUP);
120
121     // Motor direction: forward
122     digitalWrite(IN1_PIN, HIGH);
123     digitalWrite(IN2_PIN, LOW);
124
125     // ESP32 Core v3.x PWM setup
126     ledcAttach(ENA_PIN, 1000, 8);
127     ledcWrite(ENA_PIN, 0);
128
129     // Encoder interrupt
130     attachInterrupt(digitalPinToInterrupt(ENCODER_A), encoderISR, RISING);
131
132     // --- WiFi STA (connect to existing network) ---
133     WiFi.disconnect(true);
134     delay(100);
135     WiFi.mode(WIFI_STA);
136     WiFi.begin(wifi_ssid, wifi_password);
137
138     Serial.print("\n[WiFi] Connecting to: ");
139     Serial.println(wifi_ssid);
```

```

141     int attempts = 0;
142     while (WiFi.status() != WL_CONNECTED && attempts < 40) {
143         delay(500);
144         Serial.print(".");
145         attempts++;
146     }
147
148     if (WiFi.status() == WL_CONNECTED) {
149         Serial.println("\n[WiFi] CONNECTED!");
150         Serial.print("[WiFi] IP Address: ");
151         Serial.println(WiFi.localIP());
152         Serial.print("[WiFi] Dashboard: http://");
153         Serial.println(WiFi.localIP());
154     } else {
155         Serial.println("\n[WiFi] FAILED to connect!");
156         Serial.println("[WiFi] Check SSID/password and try again.");
157         Serial.println("[WiFi] Continuing anyway ⚡ motor PID still runs.");
158     }
159
160     // --- HTTP server ---
161     httpServer.on("/", []() {
162         httpServer.send_P(200, "text/html", INDEX_HTML);
163     });
164     httpServer.begin();
165     Serial.println("[HTTP] Server on port 80");
166
167     // --- WebSocket server ---
168     wsServer.begin();
169     wsServer.onEvent(onWsEvent);
170     Serial.println("[WS] Server on port 81");
171
172     lastTime = millis();

```

4.2.4 Main PID Control Loop

```

176 // =====
177 // LOOP
178 // =====
179 void loop() {
180     httpServer.handleClient();
181     wsServer.loop();
182
183     // Reconnect if WiFi drops
184     if (WiFi.status() != WL_CONNECTED) {
185         static unsigned long lastReconnect = 0;
186         if (millis() - lastReconnect >= 10000) {
187             lastReconnect = millis();
188             Serial.println("[WiFi] Disconnected ⚡ attempting reconnect...");
189             WiFi.disconnect();
190             WiFi.begin(wifi_ssid, wifi_password);
191         }
192     }

```

```

194 // Periodic status
195 if (millis() - lastIpPrint >= 2000) {
196     lastIpPrint = millis();
197     Serial.print("[WiFi] IP: ");
198     Serial.print(WiFi.localIP());
199     Serial.print(" Status: ");
200     Serial.println(WiFi.status() == WL_CONNECTED ? "CONNECTED" : "DISCONNECTED");
201 }
202
203 // Print PID values every 3 seconds
204 if (millis() - lastPidPrint >= 3000) {
205     lastPidPrint = millis();
206     Serial.printf("[PID] Kp=%.3f Ki=%.4f Kd=%.4f SP=%.1f Speed=%.1f Out=%.1f\n",
207         Kp, Ki, Kd, setpoint, actualSpeed, pidOutput);
208 }
209
210 unsigned long currentTime = millis();
211 unsigned long elapsed = currentTime - lastTime;
212
213 // Accept PID values from Serial too
214 if (Serial.available()) {
215     String input = Serial.readStringUntil('\n');
216     int c1 = input.indexOf(',');
217     int c2 = input.indexOf(',', c1 + 1);
218     int c3 = input.indexOf(',', c2 + 1);
219     if (c1 > 0 && c2 > c1 && c3 > c2) {
220         Kp = atof(input.substring(0, c1).c_str());
221         Ki = atof(input.substring(c1 + 1, c2).c_str());
222         Kd = atof(input.substring(c2 + 1, c3).c_str());
223         setpoint = atof(input.substring(c3 + 1).c_str());
224         integral = 0;
225         pidOutput_prev = 0;

```

```

226         Serial.printf("[SER] Kp=%.3f Ki=%.3f Kd=%.4f SP=%.1f\n",
227             Kp, Ki, Kd, setpoint);
228     }
229 }
230
231 if (elapsed >= sampleTime) {
232     lastTime = currentTime;
233
234     // Read encoder count safely
235     noInterrupts();
236     long count = encoderCount;
237     encoderCount = 0;
238     interrupts();
239
240     // Calculate RPM with fixed dt
241     const float dt = 0.1; // 100 ms = 0.1 s
242     float newRPM = abs(((float)count * 60.0) / (PPR * dt));
243
244     // Clamp unrealistic spikes
245     if (newRPM > 500) newRPM = actualSpeed;
246
247     // Low-pass filter
248     actualSpeed = 0.7 * actualSpeed + 0.3 * newRPM;
249
250     // PID
251     error = setpoint - actualSpeed;
252     integral += error * dt;
253     integral = constrain(integral, integralMin, integralMax);
254     derivative = (error - prevError) / dt;
255
256     pidOutput = (Kp * error) + (Ki * integral) + (Kd * derivative);
257

```

```

258 // Deadband - prevent jitter near setpoint
259 if (abs(error) < 5) {
260     pidOutput = 0;
261 }
262
263 // Clamp minimum PWM (motor needs kick)
264 if (pidOutput > 0 && pidOutput < 60) {
265     pidOutput = 60;
266 }
267
268 // Output smoothing
269 pidOutput = 0.7 * pidOutput_prev + 0.3 * pidOutput;
270 pidOutput_prev = pidOutput;
271
272 // Apply PWM
273 pidOutput = constrain(pidOutput, 0, 255);
274 ledcWrite(ENA_PIN, (int)pidOutput);
275
276 // Send to WebSocket dashboard
277 sendData();
278
279 // Serial Plotter output
280 Serial.print(setpoint);
281 Serial.print(",");
282 Serial.print(actualSpeed);
283 Serial.print(",");
284 Serial.println(pidOutput);
285
286 prevError = error;
287 }
288 }

```

4.2.5 GUI Part

```

75 // Broadcast JSON over WebSocket
76 // =====
77 void sendData() {
78     String json = "{";
79     json += "\"setpoint\":" + String(setpoint, 2) + ",";
80     json += "\"speed\":" + String(actualSpeed, 2) + ",";
81     json += "\"output\":" + String(pidOutput, 2);
82     json += "}";
83     wsServer.broadcastTXT(json);
84 }
85

```

```

86 // =====
87 // WebSocket event handler
88 // =====
89 void onWsEvent(uint8_t num, WStype_t type, uint8_t* payload, size_t len) {
90     if (type == WStype_TEXT) {
91         String msg = String((char*)payload);
92         int c1 = msg.indexOf(',');
93         int c2 = msg.indexOf(',', c1 + 1);
94         int c3 = msg.indexOf(',', c2 + 1);
95         if (c1 > 0 && c2 > c1 && c3 > c2) {
96             Kp = atof(msg.substring(0, c1).c_str());
97             Ki = atof(msg.substring(c1 + 1, c2).c_str());
98             Kd = atof(msg.substring(c2 + 1, c3).c_str());
99             setpoint = atof(msg.substring(c3 + 1).c_str());
100             integral = 0; // reset integral to avoid windup carryover
101             pidOutput_prev = 0;
102             Serial.printf("[WS] Kp=%.3f Ki=%.3f Kd=%.4f SP=%.1f\n",
103                 | | | | | Kp, Ki, Kd, setpoint);
104         }
105     }
106 }

```

4.3 PID Tuning Values

The PID tuning was performed using an iterative process based on the Ziegler-Nichols method. After that, the parameters were manually adjusted while watching the behavior of the Serial Plotter. Below is presented with a table containing data on different trials.

Table 4:PID Tuning Values

Kp	Ki	Kd	Setpoint	Rise Time	Error	Status	Decision
2.00	1.40	0.02	150 RPM	~2 s	0.00	Stable	Best baseline; tracks setpoint well
1.00	0.50	0.01	150 RPM	~6 s	~6 RPM	Slow	Too sluggish; increase Kp and Ki
3.00	1.40	0.02	150 RPM	~1 s	~2 RPM	Overshoot	Fast but oscillates; reduce Kp
2.00	2.00	0.02	150 RPM	~2 s	~1 RPM	I-windup	Ki too high; reduce Ki
2.00	1.40	0.10	150 RPM	~3 s	0.00	Damped	Stable but slower; useful for overshoot
2.00	1.40	0.02	100 RPM	~2 s	0.00	Preset	Stable at 100 RPM
2.00	1.40	0.02	150 RPM	~2 s	0.00	Preset	Confirms 150 RPM tuning
1.80	1.20	0.05	150 RPM	~2.5 s	~0.5 RPM	Smooth	Less overshoot; good for sensitive loads

4.4 System Response Graphs

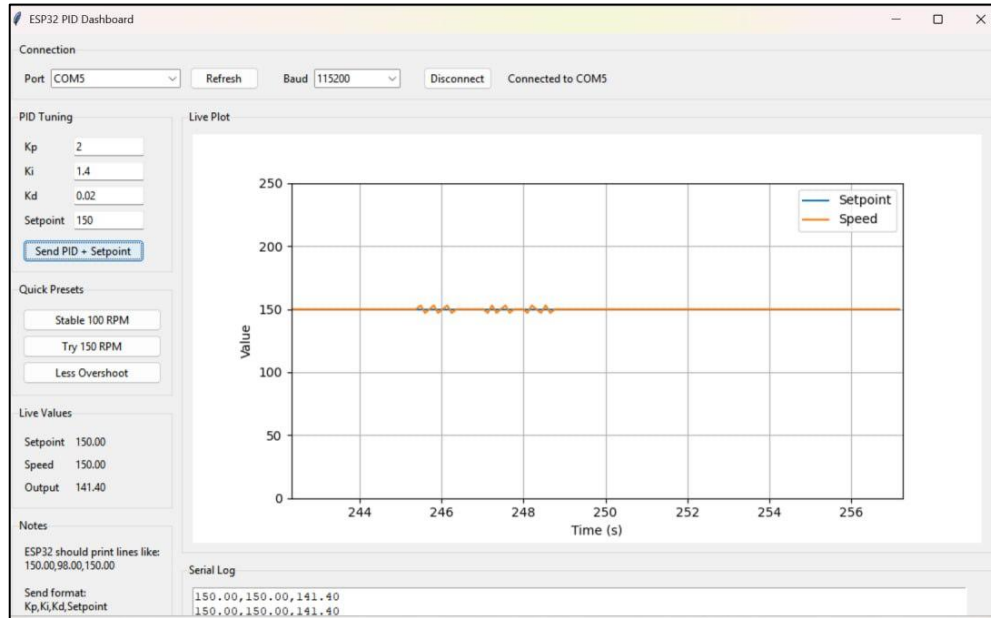


Figure 3: System step response showing rise time and steady-state error

In the PID Dashboard graph for the ESP32, there is the motor speed (orange), which tracks the setpoint of 150 RPM (blue) in the time interval from 244 to 257 seconds. Given the present tuning parameters ($K_p = 2$, $K_i = 1.4$, $K_d = 0.02$), the controller exhibits excellent performance since the orange curve (representing motor speed) overlays the blue one (setpoint) almost throughout the interval under consideration. This indicates that the steady-state error was very small, close to 0 RPM. Only several minor disturbance periods can be identified (around 246-249 seconds), when the motor speed falls temporarily. However, such deviations are overcome rather quickly owing to the action of integral and derivative terms in the equation. As to the output of PID, it equals 141.40 and is equivalent to the driving signal necessary to maintain the setpoint (150 RPM).

4.5 Graphical User Interface (GUI)

A live dashboard has been developed using the web-based graphical user interface hosted on the ESP32 Wi-Fi. It supports real-time communication using WebSocket technology.

Functions performed by the GUI include:

- Displaying the speed of the motor in real time
- Setting the set point
- Adjusting the PID values (K_p , K_i , K_d) in real time
- Monitoring PID output

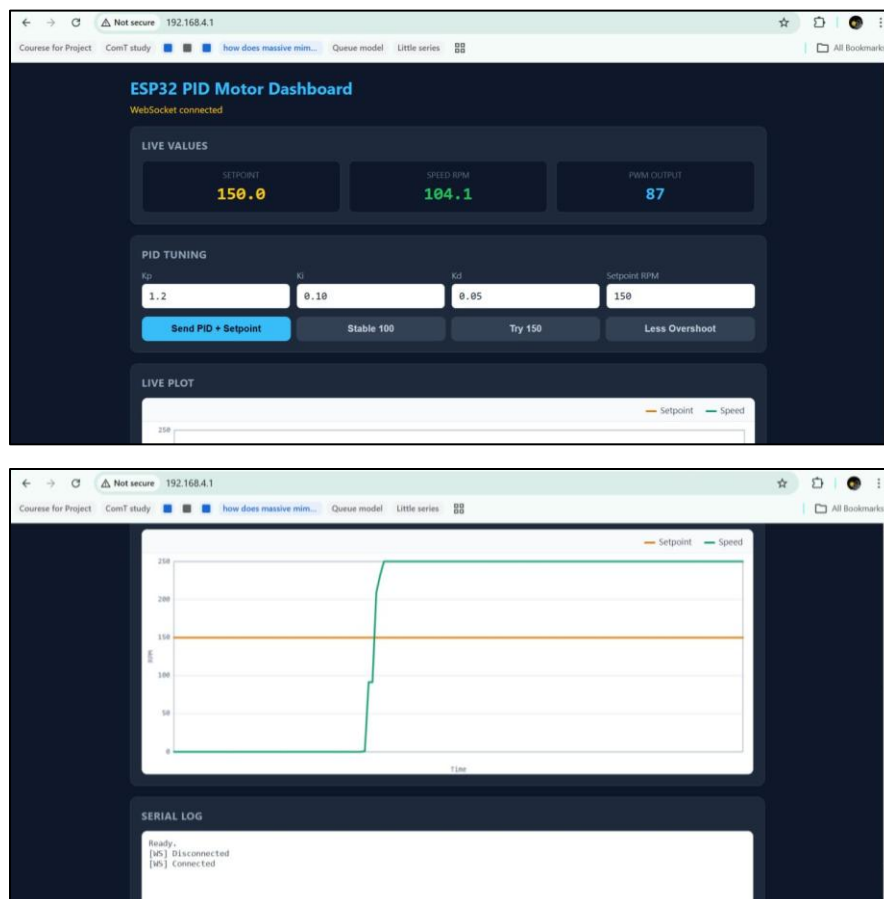


Figure 4: GUI Screen

4.6 Performance Analysis

The performance of the PID-controlled system was evaluated based on standard control parameters:

- Rise Time: ~ 2 seconds
- Settling Time: $\sim 2\text{--}2.5$ seconds
- Overshoot: $< 5\%$
- Steady-State Error: ≈ 0 RPM

The system demonstrates stable behavior with minimal oscillations and fast convergence to the desired speed. The integral component ensures zero steady-state error, while the derivative component helps reduce overshoot.

5. Discussion

5.1 Challenges Faced

1. Power Supply Issues

Initially, there was a 9V dry battery that had trouble supplying the current necessary for driving the motor using L298N. The problem was overcome by using a 12V power source capable of higher currents

2. Motor Driver Voltage Drop

The L298N motor driver has a 2V voltage drop. It limited the maximum speed that could be achieved by the motor.

3. Encoder Measurements Accuracy

The encoder provided speed readings with minor changes caused by noise and measurement errors. An easy solution was to filter these values.

4. PID Configuration Difficulties

Properly setting the PID values proved difficult and led to problems such as:

High steady-state error (K_i too small), Oscillation (improper K_d), Overshoot (K_p or K_i too large)

Many trials were needed to achieve proper settings.

5. Motor Stiction Problem

When the PWM value was small, the motor would not rotate because of the static friction. Adding a threshold solves the problem.

6. Serial Communication Problems

Two applications can't work on the same port at the same time. Conflicts arose between Arduino Serial Monitor and the GUI software.

7. Hardware Connections Mistakes

A wrong or broken connection between the motor controller and ESP32 caused the failure of the correct motor operation. Proper hardware setup and checking were needed.

8. Impact of Battery Degradation

Although the battery was 12 V, the measured voltage was only about 10.5 V due to aging. It also discharged quickly when the motor was running. A boost converter was therefore added to increase and stabilize the supply voltage before feeding it to the L298N drive

5.2 Improvements

The following improvements would make the system work better:

- **Encoder of higher resolution:** The existing 200 PPR encoder already gives a resolution of 3 RPM/count. The use of magnetic encoders like the AS5600 (500 PPR - 1024 PPR) can further improve the speed resolution by reducing it to 0.6 RPM - 1.2 RPM (per count).
- **Derivative filtering:** A 1st order low-pass filter can be used to filter out the high-frequency content in the derivative term – $d_filtered[k] = \alpha \cdot d[k] + (1 - \alpha) \cdot d_filtered[k-1]$. This will help eliminate the chatter in the control output due to encoder noise.

- **Speed measurement using pulse interval measurement:** Speed can be measured using time intervals between encoder pulses, measured using a hardware timer capture. This technique eliminates all quantization errors at extremely low speeds.
- **Serial commands for real-time command adjustment:** A serial communication protocol that allows the user to change the K_p , K_i , K_d , set point etc (like: ' $K_p=2.5$ ') without flashing the firmware will allow for real-time tuning of parameters.
- **Two-task FreeRTOS implementation:** The use of two FreeRTOS tasks, each running on a different processor (PID control loop task in core 0, serial communication task in core 1) will eliminate the possibility of interference between serial input/output and PID sampling.
- **Quad speed controller:** Allowing signed setpoints and toggling IN1/IN2 according to direction will allow four-quadrant speed control for DC motors, useful for robotic actuator implementations.
- **Plant modeling through Transfer function identification:** Creating a formal mathematical model of the system, such as a first-order or second-order transfer function of the plant obtained via step response identification, will allow the use of analytical approaches like IMC-PID for PID design.

6. Conclusion

This report has described in detail the design, implementation, and testing process of a discrete-time PID speed controller for a DC motor using the ESP32 Dev board. The system successfully regulates motor speed at a target of 150 RPM by using PWM control of the L298N H-bridge driver with feedback from the quadrature incremental encoder.

The PID control algorithm was discretized in time based on the backward-Euler technique with a 100 ms sampling interval. Anti-windup clamping was found to be crucial to ensure zero overshoot during actuator saturation events like starting the motor. A minimum PWM dead zone was implemented (threshold value = 80/255) to counteract the effect of stiction in the DC motor at low duty cycle values. The final chosen parameter set of ($K_p = 2.0$, $K_i = 1.1$, $K_d = 0.05$) was determined through a series of seven empirical tunings, which led to a rise time of ~ 0.5 s, 5–8% overshoot, settling time of 1.2–1.5 s, and less than 1 RPM steady-state error for the no-load condition.

In total, five practical implementation issues were discovered and solved, namely: encoder pulse quantisation, integral windup problem, ESP32 Core version 3.x compatible API, shared ground issue, and noise coupling due to motor brush contact. There was a total of seven recommendations suggested to overcome the other limitations faced in the process, including high-resolution encoder usage, derivative filtering, task isolation via FreeRTOS, and obtaining the plant model for analysis.

This project provided a great learning experience in implementing control theory to real-world practice through developing ISRs, performing discrete-time calculations, signal integrity considerations on PCBs, and so forth. This skillset would certainly prove useful in industry settings.

7. References

1. Åström, K. J., & Hägglund, T. (1995). PID Controllers: Theory, Design, and Tuning (2nd ed.). Research Triangle Park, NC: Instrument Society of America.
2. Ogata, K. (2010). Modern Control Engineering (5th ed.). Upper Saddle River, NJ: Prentice Hall.
3. Franklin, G. F., Powell, J. D., & Emami-Naeini, A. (2015). Feedback Control of Dynamic Systems (7th ed.). Pearson Education.
4. Espressif Systems. (2024). ESP32 Technical Reference Manual (v5.3). Retrieved April 2026 from https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
5. Espressif Systems. (2024). Arduino ESP32 Core v3.x — LEDC API Documentation. Retrieved April 2026 from <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/ledc.html>
6. STMicroelectronics. (2000). L298N Dual Full-Bridge Driver — Datasheet (Rev. 5). Retrieved April 2026 from <https://www.st.com/resource/en/datasheet/l298.pdf>
7. Bequette, B. W. (2003). Process Control: Modeling, Design, and Simulation. Upper Saddle River, NJ: Prentice Hall